

Декодирование потока по шагам — учебное пособие

Это третья, самая подробная версия руководства по декодированию потока DataShield — «для учащихся техникума»: каждый термин вводится с нуля, каждый шаг конвейера разобран на числах и трассах. Формальная монография — `DecoderGuide.Academic.ru.md`, короткий инженерный пересказ — `DecoderGuide.Engineer.ru.md` (нумерация разделов у них совпадает между собой; здесь своя, «уроковая»). Общий обзор — в `AlgorithmGuide.ru.md` (разделы 5–9); прямое преобразование — семейство `EncoderGuide.*`. Сама сборка файла из накопленного — отдельная тема, разобранная в семействе `AssemblyGuide.*`; здесь она только «вход в дверь».

Прочитав пособие, вы сможете своими словами объяснить: как из зашумлённого потока декодер выуживает пакеты, почему заголовок опознаётся сам по себе, а сектор — только вместе со «своим» заголовком, что происходит, когда сектор пришёл раньше заголовка, и зачем в конце обязательно вызывать `Complete`.

1. Словарь: слова, без которых дальше никуда

Термин	Что это на самом деле
Декодер	Класс <code>FileDecoder</code> : поток байт → накопленные слоты приёма (и, по запросу, собранный файл).
FEC-поток	Вход: Base64-текст (<code>.DataShield.txt</code>) или сырые пакеты (<code>.DataShield.bin</code>).
Конвейер	Цепочка «источник → фильтр → сканер → накопитель», соединённая событиями готовности данных.
Источник (<code>IDataSource</code>)	Даёт байты: из массива, потока или файла.
Обработчик (<code>IDataProcessor</code>)	И источник, и вход: принимает чужие байты, обрабатывает, выдаёт свои. Так строятся цепочки.
Фильтр	<code>ByteRangeFilter</code> : пропускает только символы алфавита Base64, остальное выбрасывает.
Сканер	<code>SlidingWindowScanner</code> : скользящее окно, выуживающее 75-байтные пакеты из потока.
Окно	Кусок потока фиксированной длины: 100 символов (txt) или 75 байт (bin).
Накопитель	<code>StreamProcessor</code> : собирает пакеты, создаёт слоты, раздаёт секторы по слотам.
Слот приёма	<code>ReceptionSlot</code> : всё, что принято про один файл: заголовок + версии секторов.
<code>H5</code>	24-байтный хеш заголовка; «сид», без которого сектор не проверить.
Удержанные данные	Весь поток, пропущенный сканером через себя, — он хранится на случай перепривязки.
Перепривязка	Повторная проверка удержанных данных по новому заголовку.
Прямой проход	Основное сканирование вперёд, окно за окном.
Повторный проход (rescan)	Исчерпывающая проверка удержанных данных на каждой позиции.

2. Что делает декодер: общая картина

Вход — поток байт (текст или бинарный), возможно повреждённый: с мусором, дырами, склейками. Выход — слоты приёма: по одному на каждый опознанный файл. Дальше по слоту можно попробовать собрать файл (`TryAssemble` — уже тема `AssemblyGuide.Tutorial.ru.md`). Конвейер:

источник байт (массив / поток / файл)

|

▼ (только txt-режим)

фильтр Base64: оставить A-Z a-z 0-9 + / , выбросить остальное

|

▼

сканер скользящего окна: 100 симв. (txt) или 75 байт (bin)

|

окно не опознано → сдвиг на 1; опознано → выдаём пакет, прыжок на окно

▼

накопитель StreamProcessor: пакеты → слоты приёма

|

заголовок: проверка H5 (автономно) → слот/счётчик копий

|

сектор: проверка D3 с сидом H5 → версия в слот

▼

слоты → TryAssemble (семейство AssemblyGuide)

3. Входные форматы и автоопределение

Два формата, зеркально кодеру:

Формат	Расширение	Что видит декодер
Base64	<code>.DataShield.txt</code>	строки по 100 символов, возможен мусор и декоративные обрамления
Binary	<code>.DataShield.bin</code>	сырые пакеты по 75 байт, возможны дыры и рассинхронизация

При работе с файлом (`PacketIO.ScanFile`) формат определяется по расширению: `.txt` → Base64, `.bin` → Binary, всё прочее → считается Base64. При работе с потоком (`Scan(Stream, format)`) формат задаётся явно — у потока нет расширения.

4. Конвейер на пальцах: источники и обработчики

Всё в конвейере — «источник данных» с событием `DataReady` («данные готовы — забирайте»). Обработчик — это источник, у которого есть ещё и вход: `Attach(источник)` подписывается на его данные, `Complete()` означает «вход кончился, выдай остаток». Так выстраивается цепочка любой длины.

Три практических правила:

- Запуск — каскадный.** `source.Start()` запускает всю цепочку; источник читает блок, заполняет буфер, объявляет `DataReady`; следующий модуль вычитывает и обрабатывает, пока не иссякнет.
- Завершение — по порядку.** В конце конвейер закрывают строго с головы: `filter.Complete()`, потом `scanner.Complete()`, потом `processor.Complete()`. Каждый выдаёт остаток буфера следующему.

3. **Ошибки — по каналу** **Error**. Если источник упал (сбой ввода-вывода), его свойство **Error** хранит исключение, а задача **Completion** завершается с этой ошибкой; обработчики лишь передают ошибку вверх по цепочке. Ошибка не глотаётся и не теряется.

5. Фильтр Base64: первый рубеж чистки

Txt-режим: поток — текст, и в нём бывает всё: переводы строк, пробелы, обрамления, случайный мусор. Фильтр держит таблицу на 256 значений «пропустить/выбросить» и пропускает ровно 5 диапазонов: **A-Z**, **a-z**, **0-9**, **+**, **/**. Символа **=** в таблице нет — и не нужно: пакеты кодируются в ровно 100 символов без добавки.

Результат: сканер получает плотный поток «чистых» символов Base64. В binary-режиме фильтра в цепочке нет вообще — сырые байты идут прямо в сканер.

6. Скользящее окно: как выуживаются пакеты

Сканер двигает по потоку окно фиксированного размера: 100 символов в txt, 75 байт в bin. Логика одна и та же:

```
пока хватает данных:
    показать окно обработчику
    обработчик сказал «это пакет!» → выдать пакет, прыгнуть на размер окна
    обработчик сказал «мусор»      → сдвинуться на 1 байт
```

То, что происходит внутри обработчика окна, — разделы 7–8. Важные детали:

- **Шаг 1 при неудаче** — это гарантия: начало пакета не может «проскочить» между окнами. Даже если пакеты слиплись с мусором, окно однажды встанет точно на начало.
- **Прыжок на окно при успехе** — быстрая дорога по чистому потоку: подряд идущие пакеты читаются окно за окном без перебора.
- Сканер ничего не выбрасывает: **весь поток удерживается в памяти** (зачем — разделы 10–12).

Численный пример (bin-режим): поток = 10 байт мусора + заголовок (75) + сектора. Окно в позиции 0–9 — мусор (10 сдвигов по 1); позиция 10 — заголовок опознан → пакет, прыжок на 75; далее сектора подряд. Итого: 10 неудач + серия успехов.

7. Как опознаётся заголовок

Обработчик окна получает 75 байт кандидата и решает: пакет или нет. Для заголовка проверка автономная — никаких знаний о файлах не нужно:

первые 51 байта → SHA-256 → первые 24 байта == последние 24 байта пакета?

Совпало — это корректный заголовок (вероятность случайного совпадения 2^{-192}). Накопитель распарсивает поля (имя, размер, SHA-256 файла, число ECC-томов) и создаёт слот приёма. Если такой заголовок уже известен — просто увеличивает счётчик копий заголовка.

8. Как опознаётся сектор (и почему без заголовка никак)

Сектор устроен так, что его хеш `D3` считается с сидом `H5` конкретного файла. Поэтому проверка сектора возможна только перебором известных слотов:

```
номер сектора из D1 попадает в диапазон 0..T-1 этого слота?  
D3 == Trunc9(SHA-256(H5 слота || первые 66 байт кандидата))?
```

Обе проверки прошли — сектор «свой», его payload отправляется в слот как версия (со счётчиком подтверждений — см. [AssemblyGuide.Tutorial.ru.md](#), раздел 4). Отсюда фундаментальное следствие: **пока ни одного заголовка не опознано, секторы не опознаются вовсе**. Что делать, если поток начался с середины и сектор пришёл раньше заголовка? Ответ — раздел 11.

9. Накопитель: от байтов к пакетам и слотам

`StreamProcessor` получает от сканера байты кусками произвольного размера. Края кусков не обязаны совпадать с границами пакетов, поэтому накопитель держит 75-байтный сборочный буфер `_pending`: докладывает байты, пока не наберётся полный пакет, — и только тогда обрабатывает его целиком. Пакеты никогда не режутся границами выдачи.

Принятый пакет идёт по одной из двух дорожек: заголовок → слот (раздел 7), сектор → во **все** слоты, которым он подходит (раздел 8). Принятые пакеты накопитель переправляет дальше по конвейеру неделимыми порциями.

10. Слоты приёма и версии секторов

Внутри слота — словарь «номер сектора → список версий», отсортированный по убыванию счётчика подтверждений. Повтор того же payload усиливает версию, другой payload — создаёт конкурирующую. Всё, что дальше происходит с версиями (точки ветвления, перебор, прокрутка, RS-восстановление и финальная проверка SHA-256), — предмет семейства [AssemblyGuide.*](#). Здесь достаточно знать: к моменту сборки слот — это «все кандидаты на каждый байт файла + арбитраж».

11. Проблема «сектор раньше заголовка» и адресная перепривязка

Сценарий: поток перехвачен с середины, первый пакет — сектор данных. Прямой проход его не опознает (слотов ещё нет) — окно сдвинется дальше. Потерян ли сектор навсегда? Нет:

1. Сканер **удерживает весь пропущенный поток** в памяти.
2. Когда позже опознаётся заголовок и создаётся слот, накопитель объявляет событие «принят новый заголовок».
3. Декодер просит **оба** сканера (txt и bin) перепривязать удержанные данные: по удержанным байтам выполняется повторный проход, где каждое окно проверяется уже против нового заголовка — «это не сектор ли нашего новосёла?»
4. Найденные секторы выдаются в накопитель и оседают в слоте.

Перепривязка адресная: проверяется только принадлежность к одному конкретному заголовку, счётчики остальных слотов не затрагиваются.

12. Повторное сканирование: почему без прыжков

У повторного прохода две особенности, и обе важны.

Исчерпываемость. Прямой проход после успеха прыгает на размер окна — и на повреждённом потоке прыжок может перескочить начало пакета, перекрывающегося с распознанным.

Повторный проход проверяет окно **на каждой позиции**, без прыжков: ничего не теряется по построению.

Граница. Повторный проход покрывает не весь удержанный поток, а только до позиции, где прямой проход уже выдал данные потребителю. Дальше прямой проход и так работал «со знанием» (заголовок уже был опознан), и повторная выдача задвоила бы подтверждения.

Граница фиксируется в момент выдачи партии данных — аккуратно, до вызова обработчиков.

13. Финализация: зачем **Complete** и почему нельзя забыть

В конце **Scan** декодер закрывает конвейер по порядку: фильтр → сканер → накопитель. **Complete** у каждого — «вход кончился, выдай остаток». Это не формальность: финальная выдача сканера запускает перепривязку, покрывающую пропуски прямого прохода. Кто полагается на полноту приёма — обязан дожидаться **Complete**. После этого прогресс получает финальные 100% (**Done**).

14. Прогресс и отмена

Прогресс — глобальная шкала 0–100 по потреблённым байтам: `позиция прямого прохода × 100 / размер входа`. Название фазы зависит от состояния: пока не найдено ни одного заголовка — «поиск заголовка», после — «поиск секторов». Отмена (`CancellationToken`) проверяется на каждом продвижении и в ожидании завершения конвейера; при отмене — исключение, состояние слотов остаётся как есть.

15. Сборка файла: вход в отдельное руководство

Когда поток отсканирован, у декодера спрашивают `TryAssemble(заголовок)`: он находит слот с побайтово совпадающим заголовком и вызывает сборку слота с RS-адаптером. Дальше начинается математика версий и коллизий — предмет `AssemblyGuide.Tutorial.ru.md`. Декодер отвечает только за «доставку груза на склад»; сборка — отдельный цех.

16. IO-модуль: источники, приёмники, канал ошибок

Рядом с конвейером живут вспомогательные классы:

Класс	Роль
<code>ByteArraySource</code> , <code>StreamSource</code> , <code>FileSource</code>	источники: массив, поток (не закрывается), файл
<code>StreamDataWriter</code> , <code>FileDataWriter</code> , <code>ByteListWriter</code> , <code>PreallocatedBufferWriter</code>	приёмники результата сборки
<code>IDataSource.Error</code>	канал ошибок: исключение источника; <code>Completion</code> завершается с ним

Смысл канала ошибок: сбой чтения (например, диск отвалился посреди файла) не молчит и не глотаётся — он доезжает до вызывающего кода через `Error` / `Completion`, и пользователь видит честную «ошибку ввода-вывода», а не пустой результат.

17. Сквозной пример: повреждённый txt-поток

Возьмём поток из `EncoderGuide.Tutorial.ru.md` (файл 1000 байт → `H D0...D8 H D9...D17 H`, 20 строк по 100 символов) и испортим: в строке 5 заменим один символ на другой символ алфавита Base64, перед первой строкой добавим 37 байт мусора (заголовок `>[...]`-строка).

Что произойдёт:

- Фильтр выбросит всё, кроме алфавита: декоративная строка, переводы строк, буквы мусора вне алфавита. Останутся только символы Base64.
- Окно встанет на начало чистого потока: `H` — 100 символов декодируются в 75 байт, `H5` сходится → пакет заголовка, слот создан, прыжок на 100.

3. Строки 2–5а: сектора `D0...` опознаются по `D3` с сидом `H5`, прыжки по 100.
4. Строка 5 (с порчей): 100 символов декодируются в 75 байт, но `D3` не сходится → сдвиг на 1. Окно теперь «хвост строки 5 + голова строки 6» — декодируется, но не опознаётся → ещё сдвиг... около 99 неудачных окон, пока окно не встанет на начало строки 6.
5. Строки 6–21: снова чистые прыжки по 100, включая срединный и конечный заголовки (счётчик копий заголовка дорастёт до 3).
6. `Complete`: остатки выданы, финальная перепривязка подбирает пропущенное.
7. Итог: потерян ровно один сектор (№ 4 — строка 5); при 10% ECC (`M = 2`) он восстановится кодом Рида–Соломона при сборке. Файл соберётся бит-в-бит.

Цена одного испорченного символа — ~99 лишних проверок окна: копеечная по сравнению с потерей пакета.

18. Что гарантируется

1. Пакет опознаётся только при точном совпадении хеша: мусор и подделки в слоты не попадают (2^{-72} на сектор, 2^{-192} на заголовок).
2. Ни один корректный пакет не теряется из-за мусора вокруг: шаг 1 при неудаче и исчерпывающий повторный проход это гарантируют.
3. Сектор, пришедший раньше своего заголовка, будет подобран перепривязкой.
4. Подтверждения не задваиваются: повторный проход ограничен границей выдачи.
5. Сбой ввода-вывода доезжает до пользователя через `Error`, а не маскируется.

19. Контрольные вопросы

1. Почему фильтр не пропускает `=` и почему это не страшно? (раздел 5)
2. Что происходит при неудаче распознавания окна и почему шаг именно 1? (раздел 6)
3. Почему заголовок проверяется автономно, а сектор — нет? (разделы 7–8)
4. Зачем сканер удерживает весь поток? (разделы 11–12)
5. Почему повторный проход не прыгает на размер окна? (раздел 12)
6. Что будет, если забыть вызвать `Complete`? (раздел 13)
7. Сколько неудачных окон стоит один испорченный символ Base64-строки? (раздел 17)
8. Как обрабатывается сектор, пришедший раньше заголовка? (раздел 11)
9. Куда попадает сектор, подходящий двум слотам сразу? (раздел 9)
10. Как пользователь узнает о сбое диска посреди сканирования? (раздел 16)